Manual del estudiante — Semana 2

La biblioteca: buscar en los documentos y citar

Proyecto final de Razonamiento Automático · Prof. Fabián Serrano
Asesoría: Carlos Orrego · UNAL Manizales · 12 de noviembre de 2026

La idea en una frase

Esta semana el modelo deja de responder de memoria. Antes de cada pregunta, el programa busca en los documentos del curso los pasajes que más se le parecen y se los entrega. Eso es la **configuración 2**. Como el modelo es el mismo de la semana pasada, la diferencia con la línea base mide lo que aporta la búsqueda.

Cómo usar este manual. Igual que el de la semana 1: en orden y con la terminal abierta. Las cajas azules son para hacer; las naranjas, errores típicos. Todo el código está en `semana2/taller/` del repositorio: `https://github.com/cmorregof/repo_proyecto_final`.

1. Qué cambia respecto a la semana 1

En la configuración 1 el modelo recibía el contexto de la teoría y el enunciado, y tenía que sacar la demostración de lo que recordara. Un modelo pequeño recuerda poco de Isabelle, y por eso inventa reglas y sintaxis.

En la configuración 2 se agrega un paso antes de llamar al modelo:

Paso	Qué hace
1	Una sola vez: partir el corpus en fragmentos y guardarlos en `indice.json`
2	Para cada pregunta: buscar los 3 fragmentos más parecidos (BM25)
3	Pegarlos en el mensaje, antes del enunciado, con una etiqueta [F1], [F2], [F3]
4	Llamar al **mismo** modelo y limpiar la respuesta

Todavía no hay verificación dentro del asistente: eso es la semana 3. Isabelle solo entra al final, cuando `evaluar.py` califica.

Ojo

Las tres configuraciones del informe van con el mismo modelo y la misma temperatura. Si cambian de modelo entre la 1 y la 2, la comparación ya no dice nada sobre la búsqueda.

2. Preparar

Hágalo

```
cd repo_proyecto_final
git pull                        # trae semana2/ y ConjuntoDatos/
cd semana2/taller
python3 indexar.py              # crea indice.json (unos segundos)
```

`ConjuntoDatos/` es el corpus del curso: las teorías `.thy`, los dos libros de Alonso y las diapositivas. Lo autorizó el profesor Fabián para el repositorio.

Para leer los PDF hace falta una de dos herramientas. Si no tienen ninguna, el índice se construye igual, solo con las teorías, y `indexar.py` lo avisa.

```
pip install pypdf             # opcion 1, en cualquier sistema
brew install poppler          # opcion 2, en Mac (trae pdftotext)
```

La salida esperada es parecida a esta:

```
2318 fragmentos de .../ConjuntoDatos
  pagina       1285
  lemma        728
  ...
  lemas con demostracion: 588
Listo: indice.json
```

3. Las piezas de la búsqueda

Normalizar los símbolos

En los `.thy` la conjunción se escribe `\<and>`; en los PDF y en lo que escribe el modelo, $\wedge$. Si no se unifican, la búsqueda no ve que son lo mismo. `normalizar.py` pasa los dos a una palabra común:

```
SIMBOLOS = {
    "and":     ["\\<and>", ""],
    "implies": ["\\<longrightarrow>", "", "-->"],
    "entails": ["\\<Longrightarrow>", "", "==>", "\\<turnstile>", ""],
    ...
}
```

Fragmentar por comandos, no por tamaño

Un lema partido por la mitad no le sirve a nadie. `fragmentar.py` corta cada `.thy` donde empieza un comando (`lemma`, `definition`, `fun`, `section`…) y guarda, para poder citar, el archivo, la línea y el nombre. Hay dos decisiones que vale la pena entender:

- **Los bloques `text` se pegan al lema que sigue.** En las relaciones de Alonso, el enunciado en palabras va en un `text` justo antes del lema: «Ejercicio 3. Demostrar $p \longrightarrow q$, …». Separado del lema, ese texto no ayuda a encontrar la demostración.
- **Cada lema sabe si trae demostración.** Uno que termina en `oops` o `sorry` es un enunciado sin resolver. Para demostrar no sirve, así que `config2.py` lo filtra.

Los PDF se fragmentan por página, y se citan con documento y página.

BM25

BM25 es un método clásico de recuperación por palabras. Para cada fragmento suma, por cada palabra de la consulta que aparece en él, un puntaje que crece con:

- cuántas veces aparece la palabra en el fragmento, con un techo (la décima repetición aporta poco);
- qué tan rara es la palabra en todo el corpus: `conjE` pesa mucho más que `by`.

Además penaliza los fragmentos muy largos, que contienen de todo. Son treinta líneas de Python puro en `buscar.py`, y se pueden explicar completas en la sustentación.

Se prueba solo, sin modelo:

Hágalo

```
python3 buscar.py "conjE"
python3 buscar.py "eliminacion de la disyuncion" 3
```

Por qué BM25 y no *embeddings*

Los RAG más conocidos buscan con *embeddings*, vectores que capturan el significado. Aquí empezamos por BM25 porque las consultas importantes son literales: el nombre de una regla, de una función, una fórmula. Además no hay que descargar nada. Comparar BM25 con *embeddings* locales es un buen apartado opcional del informe.

4. Medir la búsqueda antes de gastar el modelo

Si la búsqueda no encuentra el pasaje útil, el modelo no tiene con qué mejorar, y uno termina culpando al modelo de un error de la búsqueda. Por eso conviene medirla por separado.

En el benchmark público eso se puede hacer sin modelo. La solución del ejercicio n de R1 está en `R1_Deduccion_natural.thy`, en los lemas `ejercicio_n_1`, `ejercicio_n_2`... `medir_recuperacion.py` busca cada pregunta y cuenta en qué puesto aparece su solución.

Hágalo

```
python3 medir_recuperacion.py        # agreguen -v para ver cada pregunta
```

Lo que pasó en el ensayo: de 3 a 57

La primera versión de `buscar.py` usaba palabras sueltas. La solución quedó de primera en **3 de 58** preguntas y entre las tres primeras en 15. El problema es que en lógica proposicional todas las fórmulas usan las mismas palabras: `p`, `q`, `implies`, `not`. Con palabras sueltas, $p \longrightarrow q$ y $q \longrightarrow p$ son idénticas.

El arreglo fue una línea: además de las palabras sueltas, contar los **pares de palabras seguidas**.

```
def tokens(texto):
    t = re.findall(r"[a-z_][a-z0-9_']*|[0-9]+", normalizar(texto))
    return t + [a + "+" + b for a, b in zip(t, t[1:])]
```

Con eso la solución quedó de primera en **57 de 58** y entre las tres primeras en las 58.

Ojo

Parte de ese 57 viene del número: la consulta dice «Ejercicio 3» y el libro también. En el benchmark privado no habrá esa pista. Quitando el número y el nombre del lema, con la fórmula sola, la solución queda entre las tres primeras en 56 de 58. Ese es el número honesto, y es el que sirve para la formalización propia.

5. El taller: la configuración 2

De la pregunta a los pasajes

`rag.py` arma la consulta con el enunciado y el último bloque `text` del contexto, que dice el ejercicio en palabras. Busca 3 fragmentos, solo entre los lemas con demostración, y los pega con su etiqueta y su procedencia:

```
[F1] (TeoriasIsabelle/R1_Deduccion_natural.thy, linea 55 (ejercicio_1_1))
text \<open> Ejercicio 1. Demostrar ... \<close>
lemma ejercicio_1_1:
  assumes 1: "p \<longrightarrow> q" and 2: "p"
  shows "q"
proof - ...
```

Para no ahogar a un modelo pequeño, cada fragmento se corta en 1 500 caracteres. Con tres fragmentos el mensaje queda en unos 4 700 caracteres.

Un error nuevo: el modelo copia el fragmento

Con un modelo falso que copia el primer fragmento tal cual, apareció un problema. La respuesta trae el bloque `text`, la línea `lemma ejercicio_1_1:` y a veces el lema siguiente. Para `evaluar.py` eso es **trampa**, porque contiene `lemma`. La versión de la semana 1 de `extraer_prueba` solo quitaba un `lemma` al principio.

`limpiar.py` trae la versión 2: se queda con lo que va desde la primera palabra que abre una demostración (`proof`, `by`, `using`, `apply`…) hasta donde empieza otro comando. Si no encuentra ninguna, devuelve vacío. Una respuesta vacía cuenta como «sin respuesta», que es más honesto que mandar basura.

Correr y calificar

Hágalo

```
python3 config2.py 3          # primero 3, para ver que todo funciona
python3 config2.py            # las 58
python3 ../../Recursos/evaluar.py ../../Recursos/benchmark_R1.json \
        respuestas_config2.json --paralelo 4 --informe informe_c2.json
```

`config2.py` importa `modelo.py` de la semana 1, así que usa el modelo que ya tienen configurado ahí. Cada línea de `registros/config2.jsonl` guarda los pasajes recuperados con su puntaje, el mensaje completo, la respuesta cruda y la prueba limpia (requisito R5).

Lo que pasó en el ensayo con el modelo

Con `llama3.2:3b` sobre las primeras 20 preguntas de R1, calificadas con `evaluar.py`:

	aceptadas	rechazadas	trampa
Configuración 1 (sin búsqueda)	3	16	1
Configuración 2 (con búsqueda)	9	11	0

El mismo modelo triplicó lo que Isabelle acepta. Hubo dos sorpresas que vale la pena mirar en

sus propios registros. Las preguntas 13, 14 y 15 pasaban sin ayuda y fallaron con ella: los pasajes también pueden despistar. Y en la pregunta 1 el modelo tenía la solución exacta en [F1] y la copió mal, con `proof` donde el libro dice `proof -`. Tener el pasaje correcto no basta; por eso la semana 3 le devuelve al modelo el error de Isabelle.

En la configuración 1, la pregunta 6 se quedó escribiendo sin parar hasta agotar el tiempo. Desde esta semana `modelo.py` pone un tope de 800 *tokens* de salida, y una pregunta que falla se anota y no detiene la corrida. Hagan `git pull` para tenerlo.

6. Responder con fuentes, o admitir que no se sabe

Las preguntas conceptuales (R1 y R2 del enunciado) se califican de 0 a 3, y una cita inventada vale 0. `preguntar.py` usa un truco sencillo: los pasajes llegan al modelo con etiquetas, y se le pide citar *por etiqueta*. El programa, no el modelo, traduce cada etiqueta en archivo y línea o en documento y página. Si el modelo escribe una etiqueta que no existe, el código lo detecta.

Hágalo

```
python3 preguntar.py "Que diferencia hay entre rule y erule en Isabelle?"
```

La respuesta termina con la lista de fuentes realmente citadas. Hay dos avisos posibles:

- `!! CITA INVENTADA`: el modelo usó una etiqueta que no le dimos.
- `!! La respuesta no cita nada y tampoco admite que no sabe.`

Ojo

Que una etiqueta exista no garantiza que el fragmento diga lo que el modelo afirma. Eso todavía hay que revisarlo a mano, y es parte de la tarea. Es la misma lección de la semana 1: una instrucción en el prompt no es una garantía; lo que importa se revisa en código o a mano.

7. Errores típicos

Si sale: No encuentro indice.json

Falta correr `python3 indexar.py` dentro de `semana2/taller`.

Si sale: OJO: 4 PDF sin indexar

No hay `pdftotext` ni `pypdf`. Instalen uno (sección 2) y vuelvan a indexar. Para la configuración 2 basta con las teorías.

Si sale: casi todas las respuestas vacías

El modelo no escribió ninguna demostración: `limpiar.py` no encontró `proof`, `by` ni `apply`. Abran `registros/config2.jsonl` y lean el campo `crudo`. Si el modelo dice que el mensaje es muy largo, o responde sobre otra cosa, bajen `K` a 2 en `config2.py`.

Si sale: PermissionError: [Errno 13] Permission denied: ”

La variable `ISABELLE` está vacía o apunta mal. Búsquenla y expórtenla, por ejemplo `export ISABELLE=$HOME/isabelle/Isabelle2025.app/bin/isabelle`.

Si sale: ModuleNotFoundError: modelo

`config2.py` busca `modelo.py` en `../../semana1/taller`. Si movieron carpetas, hagan `git pull` o restauren la estructura del repositorio.

8. Tarea para el próximo jueves

1. Índice construido y `medir_recuperacion.py` corrido. Anoten el resultado.
2. Configuración 2 completa sobre las 58, con el mismo modelo de la configuración 1, y calificada. ¿Cuánto subió respecto a la línea base? ¿Cuántas respuestas salieron `estilo`?
3. Responder las preguntas del anexo A del enunciado con `preguntar.py` y revisar a mano cada cita: ¿el fragmento dice lo que el modelo afirma?
4. Tres preguntas trampa: una con premisa falsa, una fuera de tema y una que el corpus no responde. ¿El asistente lo admite?
5. Formalización: al menos cinco enunciados escritos. Con su archivo de soluciones pueden indexarlo (`python3 indexar.py su/carpeta`) y medir si la búsqueda encuentra sus propios lemas.